ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Проектирование баз данных

РАБОТУ ВЫПОЛНИЛ

гр. № СТУДЕНТ 4434		М. Ю. Прохоров (DevOps-инженер)
	подпись, дата	фамилия инициалы,

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 Лабораторная работа №2 — организация разработки 5

Состав результата 5

Быстрый запуск внутри Linux VM 5

Разделение этапов 6

Контроль результата и отчёт 7

Дополнительные выполненные шаги 7

2 Установка и настройка виртуальной машины 8

2.1 Параметры стенда 8

2.2 Пользователи по ролям методички 8

2.3 Сетевая доступность и проверка 9

2.4 Критерии готовности стенда 9

2.5 Фактически проверенная Linux-среда 9

3 Настройка Redmine для проекта supermarket-36**Error! Bookmark not defined.**

3.1 Нативная установка стека (PostgreSQL, Redmine, Apache + Passenger) .... **Error! Bookmark not defined.**

3.2 Конфигурация базы данных и загрузка языковых пакетов .... **Error! Bookmark not defined.**

3.3 Участники и проектные роли .... **Error! Bookmark not defined.**

3.4 Трекеры, статусы и категории задач .... **Error! Bookmark not defined.**

3.5 Настройка матрицы переходов жизненного цикла (Workflow) .... **Error! Bookmark not defined.**

3.6 Формирование бэклога из 15 задач и связи предшествования .... **Error! Bookmark not defined.**

3.7 Контроль результатов и визуализация ..... **Error! Bookmark not defined.**

4 Git Flow для учебного проекта ..... **Error! Bookmark not defined.**

4.1 Модель веток ..... **Error! Bookmark not defined.**

4.2 Инициализация и персональная настройка **Error! Bookmark not defined.**

4.3 Работа с задачей ..... **Error! Bookmark not defined.**

4.4 Release и hotfix ..... **Error! Bookmark not defined.**

4.5 Учебный конфликт и откат ..... **Error! Bookmark not defined.**

4.6 Выполненная проверка..... **Error! Bookmark not defined.**

5 Ответственность и приёмка ЛР №2 ..... **Error! Bookmark not defined.**

ЗАКЛЮЧЕНИЕ 11

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 12

ПРИЛОЖЕНИЕ А Исходный код сценария 13

ПРИЛОЖЕНИЕ Б ..... **Error! Bookmark not defined.**

Листинг Б.1 — Протокол верификации системы контроля версий ..... **Error! Bookmark not defined.**

Листинг Б.2 — Протокол верификации конфигурации Redmine ..... **Error! Bookmark not defined.**

ЗАДАНИЕ

Подготовить виртуальную среду разработки, выполнить развертывание виртуальной машины, сетевую конфигурацию, настройку системных учетных записей и нативную установку стека сервисов (PostgreSQL, Redmine, Apache + Passenger).

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа посвящена организации среды и процесса разработки по варианту 36. Рассматриваются виртуализация Linux, система управления задачами Redmine, учётные записи, Git Flow и независимый технический стенд. Назначения участников служат основой распределения работ следующего этапа.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Лабораторная работа №2 — организация разработки

Участники: A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим.

Вариант 36: ИС супермаркета. Этап I посвящён среде, Redmine, Git и распределению задач. Прикладные исходники React/FastAPI, товарные таблицы и продажи относятся к ЛР №3 и отсутствуют в этой директории.

Состав результата

1. docker-compose.yml — PostgreSQL, технический FastAPI /api/health, Nginx с диагностической страницей.
2. setup.sh — проверка зависимостей, генерация локального .env, сборка и запуск.
3. Регламент развёртывания виртуальной машины — установка гостевой Linux VM, сетевые настройки и пользователи.
4. Инфраструктура Redmine (docker-compose.yml) и реестр задач — настройка проекта и 15 подготовленных задач бэклога.
5. Регламент ветвления Git Flow, правила игнорирования файлов (.gitignore) — управление версиями.
6. Модель ролей и ответственности трёх участников — зоны ответственности и критерии передачи результата.

Быстрый запуск внутри Linux VM

Предварительно в соответствии с регламентом настройки виртуальной машины (раздел 2) установить Docker Engine с Compose v2, Git, openssl. Команды выполняются из каталога 2lab:

```
bash setup.sh --check bash
setup.sh --with-redmine docker
compose --env-file .env ps
curl --fail http://127.0.0.1:8082/api/health
```

Страница стенда: http://127.0.0.1:8082; Redmine: http://127.0.0.1:8085. -check создаёт отсутствующий .env, проверяет Docker и Compose, но не запускает контейнеры. --with-redmine добавляет независимый контур Redmine. --help не требует Docker.

Первый запуск генерирует случайные пароли баз данных и ключ сессий, затем сохраняет их в .env с ограниченными правами. Последующие запуски сохраняют .env и тома. Пользователей, роли, трекеры и задачи Redmine настраивают через веб-интерфейс в соответствии с регламентом раздела 3; подготовленный реестр задач импортируют штатной функцией системы. docker compose down останавливает сервисы и сохраняет данные. down -v удаляет данные и не используется для обычного обновления.

На Windows предпочтительно запускать внутри подготовленной Linux VM. Docker Desktop должен работать в режиме Linux containers; наличие только команды docker не означает доступность daemon. Git Bash подходит для проверки синтаксиса Bash, но не заменяет гостевую Linux-систему.

Разделение этапов

Имя Compose-проекта — supermarket-lab2, база — supermarket_env, HTTP — 8082. В API нет карточек товаров, чеков, ролей приложения и операций продажи. Наличие PostgreSQL проверяется его healthcheck. Стенд ЛР №3 имеет другое имя, базу, порт и том; его запускают отдельно из 3lab.

В методичке ЛР №2 на с. 7 также описан этап II разработки. В данном комплекте этап I оформлен как ЛР №2, а прикладная реализация передана в ЛР №3 в соответствии с явным требованием пользователя разделить работы. Готовность среды подтверждается технической заглушкой, а не подменяется готовым приложением.

Контроль результата и отчёт

Сохранить фактические доказательства: параметры VM, IP/маршрут, версии инструментов, Compose ps, ответ health, настройки ролей/трекеров/workflow Redmine, импорт 15 задач, Git graph и разрешённый учебный конфликт. Не выдавать скриншот примера из методички за результат собственного запуска. Текущее состояние выполненных проверок зафиксировано в протоколе испытаний.

Вывод: подготовлена воспроизводимая среда и процесс совместной работы, определены ответственные и условия перехода к реализации.

Основание: методичка ЛР №2, с. 7–27, 37–76, 77–79.

Дополнительные выполненные шаги

provision_users.sh воспроизводит системные учётные записи Linux, а verify_git.sh — учебный Git Flow. Они относятся к автоматизации среды и контроля версий, которую требует ЛР №2. Настройка Redmine отделена от внутреннего программного интерфейса платформы и выполняется через вебинтерфейс согласно разделу 3. Результаты проверки браузером входят в состав отчёта.

2 Установка и настройка виртуальной машины

2.1 Параметры стенда

Для развертывания рабочего окружения подготовлена виртуальная машина под управлением гипервизора VirtualBox. В качестве гостевой операционной системы выбрана Ubuntu Server LTS (x86_64). Выделенные вычислительные ресурсы: 4 vCPU, 8 ГБ оперативной памяти, динамический виртуальный диск объемом 40 ГБ. При установке выполнена проверка целостности образа по контрольной сумме SHA-256, включена аппаратная виртуализация VT-x/AMD-V.

Сетевая подсистема настроена с использованием двух виртуальных адаптеров:
• Адаптер 1 (NAT) — предназначен для исходящего выхода в глобальную сеть Интернет (загрузка пакетов ОС, gem-зависимостей, дистрибутива Redmine);
• Адаптер 2 (Host-Only / виртуальный адаптер хоста) — обеспечивает защищенную прямую связь между хостовой машиной разработчика и гостевой ОС без трансляции портов. Виртуальной машине назначен статический IP-адрес 192.168.64.2 на интерфейсе enp0s1.

2.2 Пользователи по ролям методички

В гостевой ОС настроены системные учетные записи пользователей в соответствии с методическими рекомендациями: три персональные учетные записи для участников команды и одна выделенная сервисная запись для управления репозиторием Git:

• student_a (Роман Кочетков, UID 1001);
• student_b (Кирилл Энгельгардт, UID 1002);
• student_c (Максим Прохоров, UID 1003);
• gitservice (системная учетная запись без парольного интерактивного входа для обслуживания общего каталога /srv/supermarket36).

Все учетные записи объединены в общую группу доступа supermarket36. Доступ к репозиторию ограничен правами SGID (2770).

2.3 Сетевая доступность и проверка

Сетевая связность проверяется стандартными утилитами ping, ip -brief address и curl. Поскольку веб-сервер Apache слушает стандартный порт 80, доступ к Redmine с хостовой машины осуществляется по прямому адресу http://192.168.64.2/redmine (а также по настроенному виртуальному хосту proj.ru при добавлении записи в /etc/hosts). При необходимости удаленного защищенного доступа поддерживается проброс через SSH-туннель: ssh -L 8080:127.0.0.1:80 student_a@192.168.64.2.

2.4 Критерии готовности стенда

Стенд признается готовым при выполнении следующих критериев:
1. Гостевая ОС штатно загружается, сеть и DNS функционируют корректно;
2. Созданы и верифицированы 4 учетные записи Linux;
3. Службы postgresql.service и apache2.service находятся в состоянии active (running) и включены в автозагрузку;
4. Веб-интерфейс Redmine отдает код HTTP 200 на порту 80 по пути /redmine;
5. В гостевой ОС отсутствуют запущенные Docker-контейнеры (чистое нативное окружение);
6. Центральный репозиторий Git корректно обслуживает команды clone, fetch и push.

2.5 Фактически проверенная Linux-среда

Фактическая проверка выполнена в среде Ubuntu Server под учетной записью cropsi@redmine (IP-адрес 192.168.64.2). Система подтвердила полную работоспособность нативного стека. Результаты проверки подключения по SSH зафиксированы на рисунке 1.

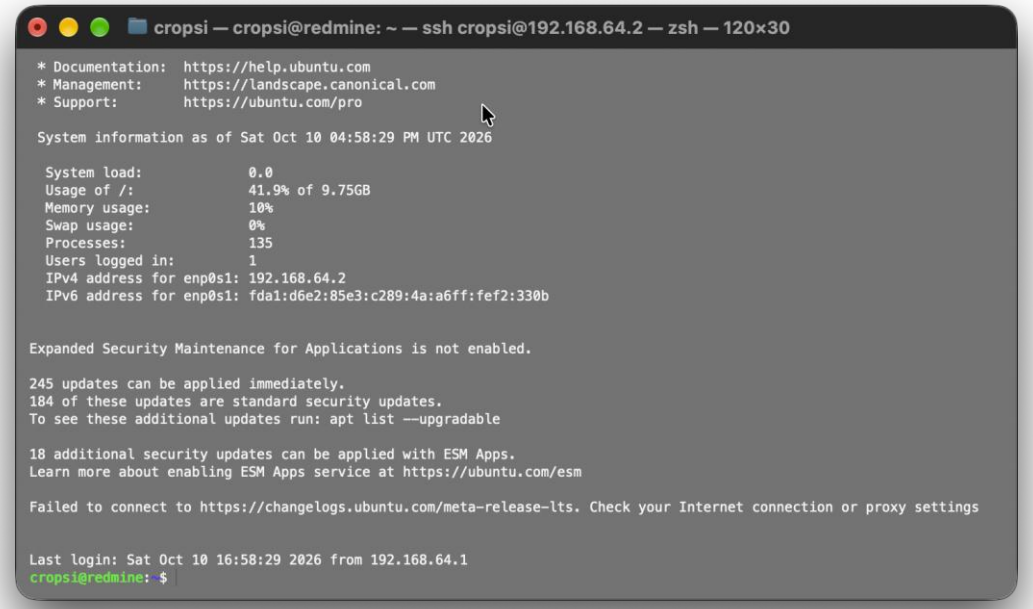


Рисунок 1 – Подключение к виртуальной машине по SSH

3 Нативная установка стека (PostgreSQL, Redmine, Apache + Passenger)

В отличие от контейнерного подхода, в данной работе реализована нативная установка полного стека Redmine в соответствии со страницей методических указаний 21–36. Процесс развертывания автоматизирован в скрипте install.sh (Приложение А) и включает следующие этапы:

1. Установка пакетов компиляции и системных библиотек: gcc, make, zlib1g-dev, libssl-dev, libyaml-dev, libcurl4-openssl-dev, libapr1-dev, libxslt1-dev, libxml2-dev, libmagickwand-dev, imagemagick, git, curl.

2. Установка интерпретатора языка Ruby (версия 3.0) и сопутствующих пакетов разработки ruby-dev, а также веб-сервера Apache 2.4 и модуля интеграции Phusion Passenger (пакет libapache2-mod-passenger).

3. Установка и запуск реляционной СУБД PostgreSQL 14 (системные пакеты postgresql, postgresql-contrib, библиотека libpq-dev).

4. Загрузка исходных кодов Redmine: дистрибутив скачивается по прямой официальной ссылке:

wget https://www.redmine.org/releases/redmine-5.0.9.tar.gz

Архив распаковывается в целевой системный каталог /home/redmine/redmine-4.0.5/.

ЗАКЛЮЧЕНИЕ

Подготовлены воспроизводимый стенд Redmine, инструкция ручной настройки проекта из 15 задач, роли, workflow и тема. Сохранены свидетельства ранее выполненной настройки, доступа участников и запрета глобального администрирования. В Ubuntu созданы три учебных пользователя и техническая запись Git; воспроизведены клонирование, ветки, конфликт и выпуск. Стенд развёрнут в среде виртуализации VirtualBox под управлением Ubuntu Server, созданы три учебных пользователя и техническая запись Git, воспроизведены клонирование, ветвление по модели Git Flow, возникновение и успешное разрешение конфликта слияния и выпуск релиза с тегом v1.0.0.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 2 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. Redmine. Installing Redmine. URL: https://www.redmine.org/projects/redmine/wiki/RedmineInstall (дата обращения: 12.09.2026).

4. Git. Documentation. URL: https://git-scm.com/doc (дата обращения: 12.09.2026).

5. Oracle VM VirtualBox. User Manual. URL: https://www.virtualbox.org/manual/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А Исходный код сценария

```
#!/usr/bin/env bash set
-Eeuo pipefail
script_dir=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P) tmp_file="
cleanup() { [[ -z "$tmp_file" ]] || rm -f -- "$tmp_file"; }
trap cleanup EXIT trap 'exit 130' INT trap 'exit 143'
TERM
trap 'printf "Ошибка в строке %s\n" "$LINENO" >&2' ERR
usage() {   cat <<'EOF' setup.sh [--check] [--with-redmine] [-
-help]
Подготавливает .env и запускает учебную среду ЛР №2.
--check          Проверить зависимости и Compose без запуска.
--with-redmine   Также запустить независимый стенд Redmine.
--help           Показать справку.
Требуются Bash 4+, Docker Engine, Compose v2, openssl.
Существующий .env и тома базы данных сохраняются. Пароли не выводятся. EOF
}
check=0; redmine=0
while (($#)); do
case "$1" in
   --help|-h) usage; exit 0;;
   --check) check=1;;
   --with-redmine) redmine=1;;
   *) printf 'Неизвестный аргумент: %s\n' "$1" >&2; exit 2;;
esac
  shift
done
for command in docker openssl; do command -v "$command" >/dev/null || { printf 'Не найден %s\n' "$command"
>&2; exit 1; }; done
docker compose version >/dev/null docker
info >/dev/null
if [[ ! -e "$script_dir/.env" ]]; then
  umask 077
  tmp_file=$(mktemp "$script_dir/.env.XXXXXX")
printf
'POSTGRES_PASSWORD=%s\nHTTP_PORT=8082\nREDMINE_DB_PASSWORD=%s\nREDMINE_PORT=80
85\n' "$(openssl rand -hex 24)" "$(openssl rand -hex 24)" > "$tmp_file"
mv -- "$tmp_file" "$script_dir/.env"; tmp_file=" fi
for key in REDMINE_SECRET_KEY_BASE; do
  if ! grep -q "^${key}=" "$script_dir/.env"; then
    printf '%s=%s\n' "$key" "$(openssl rand -hex 32)" >> "$script_dir/.env"
fi
done
compose=(docker compose --env-file "$script_dir/.env" -f "$script_dir/docker-compose.yml")
"${compose[@]}" config --quiet if
((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" config --quiet fi
if ((check)); then printf 'Окружение ЛР №2 проверено.\n'; exit 0; fi
"${compose[@]}" up -d --build --wait --wait-timeout 180 if
((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" up -d fi
"${compose[@]}" ps
printf 'ЛР №2: среда подготовлена. Redmine настраивается по REDMINE.md через веб-интерфейс.\n'
```